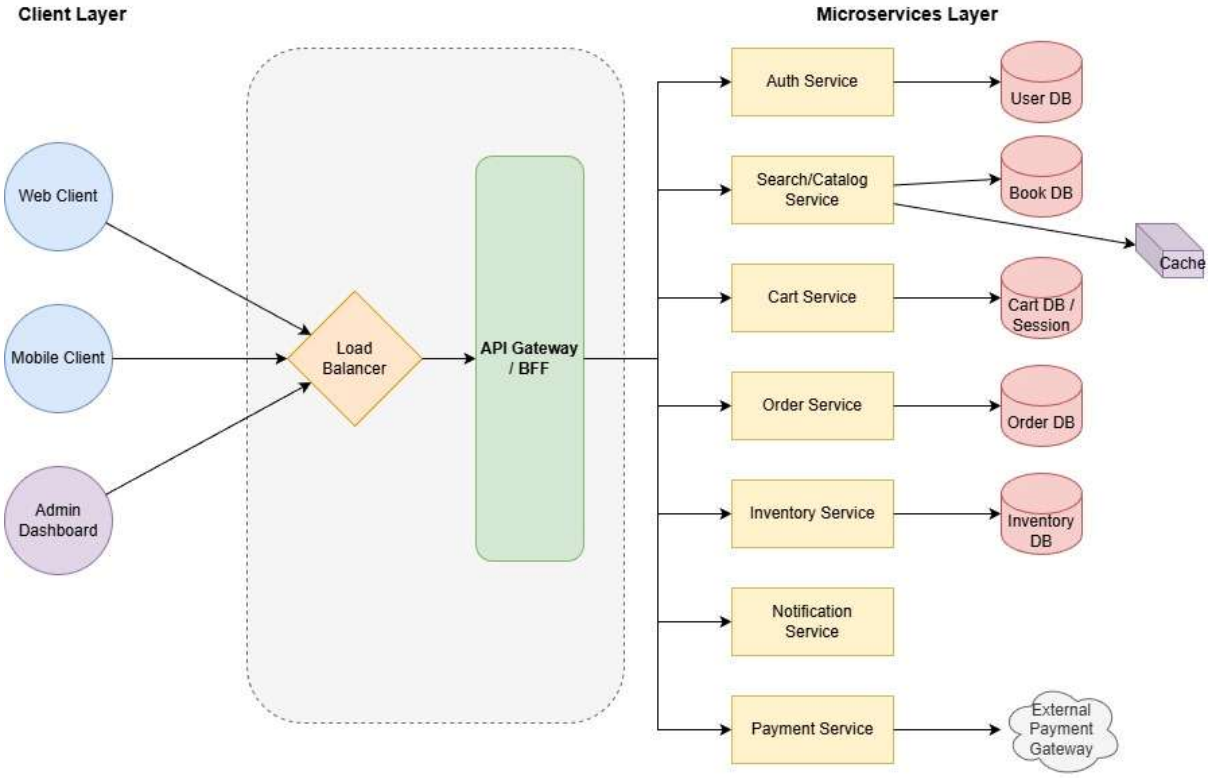
SmartBooks: An Online Bookstore System

1. Short System Description

SmartBooks is a comprehensive online e-commerce platform that enables customers to browse, search, and purchase books securely via web and mobile interfaces. The system allows administrators to manage inventory and suppliers while ensuring high availability, scalability for peak traffic, and real-time order tracking.

2. Architecture Model Diagram



---

3. Backend Architecture Choice

Selected Style: Microservices Architecture

Justification: We have selected a Microservices Architecture for SmartBooks to satisfy the high scalability and reliability requirements identified in the expert interview.

- **Scalability:** The interview highlighted that the system must handle "thousands of users" during peak sales. Microservices allow us to scale specific resource-intensive services (like the **Search/Catalogue Service**) horizontally without scaling the entire application.
- **Fault Isolation:** If the **Recommendation Service** or **Notification Service** fails, it should not crash the entire checkout process. Microservices ensure that failures are isolated, maintaining the system's overall availability.

- **Decentralized Data Management:** As seen in our Deliverable 2 Sequence Diagrams, different services require different data structures (e.g., Cart uses Session Store, Catalogue uses Caching). Microservices allow each service to manage its own database optimized for its specific needs.

4. Frontend Architecture Choice

Selected Style: Progressive Web Application (PWA) implemented as a Single Page Application (SPA)

Justification: We have selected a PWA architecture to meet the client's requirement for a system that is "accessible on both web and mobile" and supports "real-time status updates and notifications".

- **App-Like Experience:** PWA allows the web application to be installed on mobile devices without developing a separate native app, providing a seamless experience across desktop and mobile.
- **Notifications:** The client specifically requested order updates. PWA Service Workers enable push notifications even when the browser is closed, ensuring customers are updated on shipping status without having the app opened.
- **Responsiveness (SPA attributes):** By implementing this as an SPA (Single Page Application), we ensure that only necessary data is fetched from the server after the initial load. This reduces bandwidth usage and provides the "modern, responsive" interface as per the client's request.

5. Architecture Description

The SmartBooks system utilizes a **Microservices Architecture** coupled with a **Back-End for Front-End (BFF)** pattern to manage communication between the client and services.

- **Client Layer:** The Frontend is a PWA that caches assets using Service Workers for offline catalogue browsing. It communicates with the backend via APIs.
- **API Gateway / BFF:** This layer acts as the single-entry point for all client requests. It aggregates responses from multiple microservices (e.g., fetching book details + recommendations + availability) and returns a unified response to the client, reducing round-trips and frontend complexity.
- **Load Balancer:** A load balancer sits in front of the Gateway to distribute incoming traffic evenly, ensuring no single instance is overwhelmed during peak sales.
- **Microservices Layer:**
    - **Auth Service:** Handles registration, login, and token generation.
    - **Search/Catalogue Service:** Manages book data and interacts with a read-optimized cache for fast search results.
    - **Cart Service:** Manages the user's shopping session. It stores temporary item selections in a high-performance Session Store (or Cart DB), allowing users to maintain persistent cart states across different devices before checkout.
    - **Order & Inventory Services:** The Order Service handles the checkout lifecycle. It communicates directly with the Inventory Service via HTTP APIs to

check availability and reserve stock before confirming the purchase. This ensures immediate data consistency during the transaction.

- **Notification Service:** Listens for order status changes and triggers emails or push notifications via the PWA Service Worker.
- **Payment Service:** Securely processes financial transactions. It serves as a secure abstraction layer that communicates with the External Payment Gateway, ensuring sensitive data is encrypted and isolated from the rest of the system